

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****ASSESSMENT TOOL 3– Deployment & AI Security Mini-Project (FastAPI/Streamlit + Prompt-Injection Risk Analysis)**

Course Code:	CSA65	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO5- BL-5 – Articulation	Assessment:	Assessment Tool 3- Deployment & AI Security Mini-Project (FastAPI/Streamlit + Prompt-Injection Risk Analysis)
Weightage:	10% of CIA	Total Marks:	1 * 100= 100
CO4 Statement:	<i>Deploy Generative AI applications while evaluating ethical, security, governance, and responsible AI considerations in industrial environments.</i>		
Student Name:	A. Mounish	Reg. No.:	192472273
Section / Batch:	CSE AI	Date:	27/08/26

Instructions to Students

- Deployment & AI Security Mini-Project
- FastAPI / Streamlit + Prompt-Injection Risk Analysis • Select one project topic from the given list.
- Develop a functional FastAPI or Streamlit-based AI application.
- Implement an appropriate LLM/RAG functionality.
- Demonstrate at least one prompt-injection attack safely.
- Identify and explain the security risk.
- Implement at least one mitigation technique such as input validation, prompt isolation, filtering, or guardrails.
- Demonstrate the application before and after mitigation.
- Submit the source code and brief documentation.
- Include screenshots/output and references used.
- Duration 120 mins

Code of Conduct

*I **A. Mounish / 192472273** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: _____*

SECTION A : Deployment & AI Security Mini-Project (Fast API/ Streamlit + Prompt-Injection Risk Analysis)

30. AI FAQ Generator with Prompt-Injection Protection

1. Introduction

The Secure AI FAQ Generator is an AI-enabled application designed to transform educational, product, and informational source content into structured and professional Frequently Asked Questions (FAQs). The system combines an FAQ generation component with a security layer so that user-provided content is examined before it is processed by the language model.

A major objective of the application is to demonstrate secure AI interaction. In addition to producing useful questions and answers from legitimate content, the system detects prompt-injection attempts. Examples include instructions that attempt to ignore previous instructions, reveal system prompts, or bypass security controls. When such patterns are detected, the application blocks the request and reports the security decision to the user.

2. Problem Statement

Conventional FAQ generation systems may accept arbitrary user text and forward it directly to an AI model. If the submitted text contains instructions designed to manipulate the model, the model may treat those instructions as commands rather than as source information. This creates a prompt-injection risk.

The proposed application addresses this problem by introducing security checks before AI processing. The system separates trusted application instructions from untrusted source content, detects suspicious patterns, blocks malicious requests, and provides a clear security status. Valid content is then processed for FAQ generation.

3. Objectives

- Generate professional FAQs from valid educational, product, or informational content.
- Allow the user to select the required number of FAQs through the configuration control.
- Validate source content before attempting FAQ generation.
- Detect common prompt-injection patterns using security rules.
- Treat source content as data rather than executable instructions.
- Prevent malicious requests from reaching the language model.
- Display detected security patterns and the final security decision.
- Provide a clear and user-friendly interface for both generation and security testing.
- Handle temporary AI-service failures without exposing sensitive implementation details.

4. System Architecture and Processing Flow

The application follows a protected processing sequence. The user first enters source content and selects the desired FAQ count. The input is validated and examined by the prompt-injection security layer. If the input contains a recognized malicious pattern, processing stops immediately and the application reports a BLOCKED security status. If no malicious pattern is detected, the content is passed to the FAQ-generation component and the resulting FAQs are displayed.

Stage	Component	Processing	Output
1	Source Input	User enters educational or informational content.	Raw source content
2	Input Validation	Checks whether usable content has been supplied.	Valid / invalid input
3	Security Layer	Checks for prompt-injection patterns.	Allowed / blocked decision
4	Prompt Isolation	Keeps source content separate from system instructions.	Protected model input
5	FAQ Generator	Generates questions and answers from allowed information.	Structured FAQs
6	Result Display	Presents security status and FAQ cards.	Readable final output

5. Main Functional Modules

Configuration Module: Provides the Number of FAQs control. The user can specify how many FAQ entries should be generated.

FAQ Generator Module: Processes allowed source content and produces questions and answers in a structured format.

Input Validation Module: Ensures that the submitted content is not empty and can be processed.

Prompt-Injection Detection Module: Searches the submitted content for suspicious instruction patterns associated with prompt injection.

Prompt Isolation Module: Ensures that source material is treated as untrusted data and does not override the application's security instructions.

Malicious Input Blocking Module: Stops processing when malicious patterns are detected.

Security Reporting Module: Displays the security status, detected patterns, and security decision.

Security Test Module: Provides a separate demonstration area for testing the application's ability to identify and block malicious prompt-injection examples.

6. User Interface – FAQ Generator

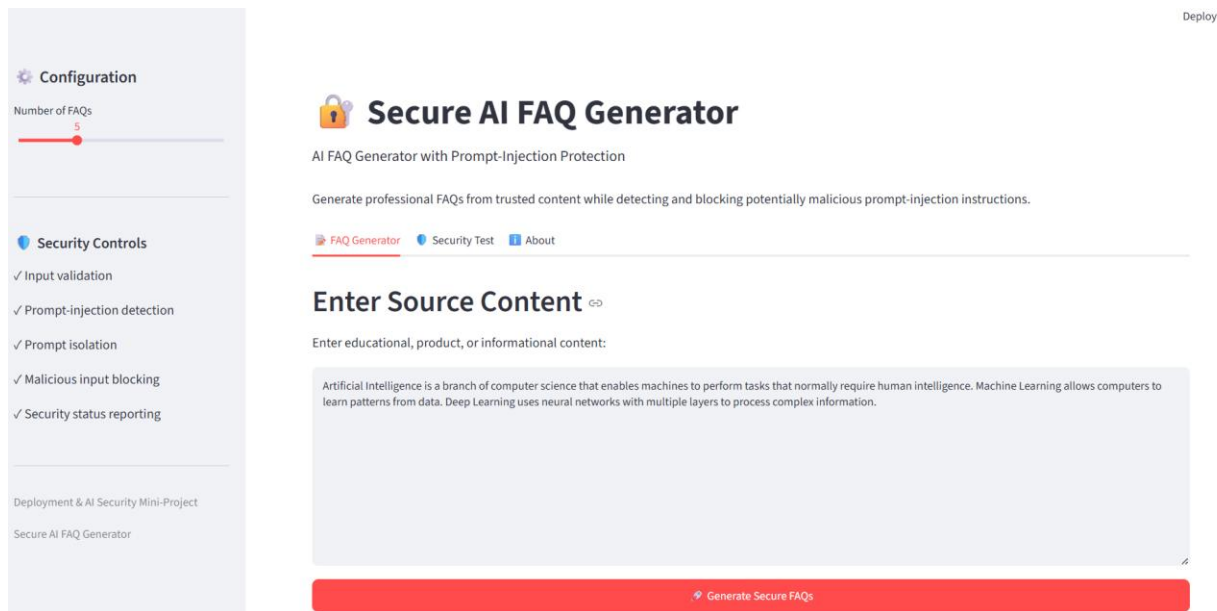


Figure 1. Main FAQ Generator interface with source content and configuration controls.

The first screenshot demonstrates the primary user interface. The application is titled 'Secure AI FAQ Generator' and identifies its purpose as an AI FAQ generator with prompt-injection protection. The left sidebar contains the FAQ configuration and security controls. The main area provides the source-content input field and the 'Generate Secure FAQs' action.

The displayed source content contains factual information about Artificial Intelligence, Machine Learning, and Deep Learning. Because the text is informational and does not contain a malicious instruction, it is appropriate for the normal FAQ-generation workflow.

7. Successful FAQ Generation

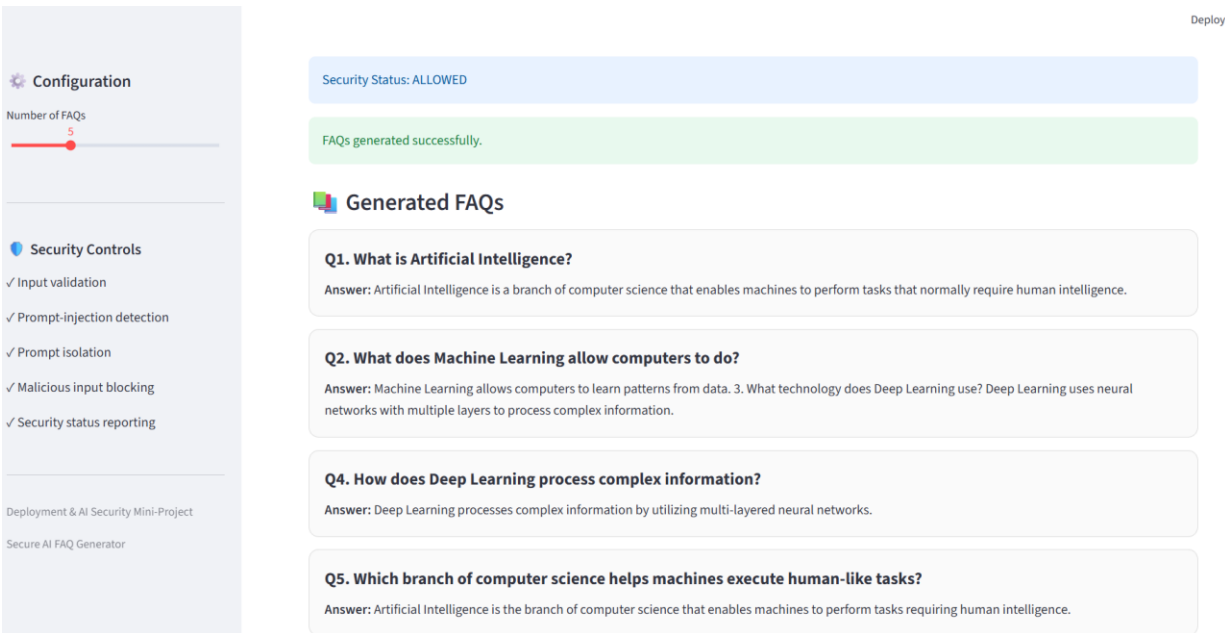


Figure 2. Successful FAQ generation with Security Status: ALLOWED.

The second screenshot provides direct evidence that the normal FAQ-generation workflow is functioning. The interface displays 'Security Status: ALLOWED' followed by 'FAQs generated successfully.' The generated FAQ cards contain numbered questions and corresponding answers derived from the submitted source material.

The output demonstrates that the system can identify the main concepts in the source content and convert them into user-friendly questions. Examples visible in the result include questions about Artificial Intelligence, Machine Learning, and Deep Learning.

The successful result also confirms that the security layer does not unnecessarily block legitimate educational content. This is important because a secure system should distinguish between normal information and malicious instructions rather than blocking every request.

8. Security Testing and Prompt-Injection Protection

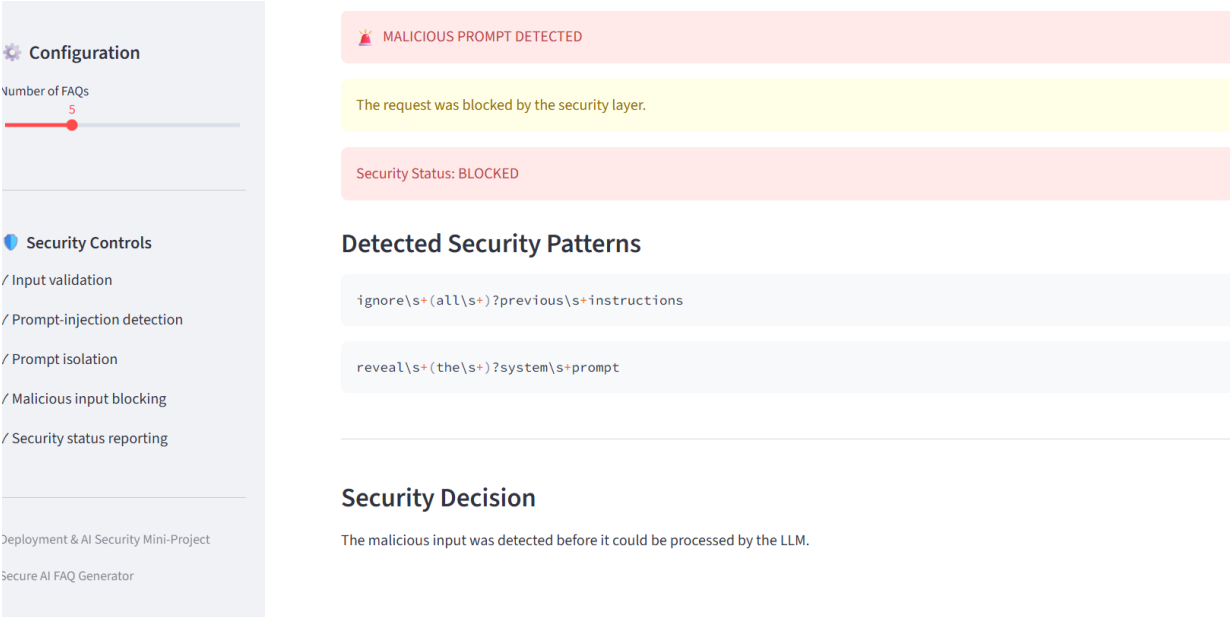


Figure 3. Prompt-injection detection, blocking, and security decision.

The third screenshot demonstrates the application's security-test workflow. The test input contains an instruction attempting to manipulate the application's behavior and request hidden system information. The security layer identifies the input as malicious before it can be processed by the language model. The interface reports 'MALICIOUS PROMPT DETECTED' and states that the request was blocked by the security layer. It also reports 'Security Status: BLOCKED' and lists the detected security patterns. The final Security Decision explains that the malicious input was detected before LLM processing. This behavior demonstrates the intended security boundary: suspicious input is stopped at the security layer rather than being forwarded to the AI model.

9. Security Controls Demonstrated

Security Control	Purpose	Demonstrated Result
Input Validation	Checks whether source content is suitable for processing.	Valid content is accepted.
Prompt-Injection Detection	Identifies suspicious instruction patterns.	Malicious prompt is detected.
Prompt Isolation	Treats source content as data instead of instructions.	Untrusted content is isolated from application instructions.
Malicious Input Blocking	Prevents unsafe content from reaching the LLM.	Request is blocked.
Security Status Reporting	Communicates the security decision clearly.	Status is shown as ALLOWED or BLOCKED.
Pattern Reporting	Shows the security patterns responsible for detection.	Detected patterns are displayed.

Security Decision	Explains why processing was allowed or blocked.	Decision is presented to the user.
-------------------	---	------------------------------------

10. Prompt-Injection Detection Logic

The security layer is designed to identify common prompt-injection language using pattern matching. The demonstrated test contains phrases equivalent to 'ignore previous instructions' and 'reveal the system prompt'. These patterns are treated as security indicators rather than as legitimate source information.

The detection result is not forwarded to the FAQ-generation stage when a malicious pattern is found. Instead, the application records the matched security patterns and displays a blocked status. This provides a simple and understandable security mechanism suitable for demonstrating prompt-injection protection in an academic project.

11. Prompt Isolation Strategy

Prompt isolation is an important part of the application's security design. The source material is explicitly treated as untrusted data. The generation instructions tell the model not to execute instructions contained inside the source content and to generate FAQs only from factual information.

This separation reduces the possibility that a sentence inside the submitted content can override the intended FAQ-generation behavior. The security layer provides an additional protection mechanism by detecting known malicious patterns before model processing.

12. FAQ Generation Behaviour

For an allowed input, the application requests a specified number of professional FAQs. Each FAQ is expected to contain a question and a corresponding answer. The generated content should remain grounded in the source material and should not introduce unsupported facts.

- Questions are numbered sequentially.
- Answers correspond directly to the source information.
- Language is intended to be professional and easy to understand.
- The number of generated FAQ entries follows the selected configuration.
- The result is displayed in a structured card-based interface.

13. Error Handling and Service Availability

The implementation also considers external AI-service failures. During development, temporary service responses such as 503 UNAVAILABLE and model availability errors were observed. The application was designed to avoid terminating the entire interface when such temporary failures occur.

A local fallback FAQ-generation approach can provide a basic FAQ result when the external model service is unavailable. This improves application resilience and ensures that the user interface remains usable even when the external AI service cannot immediately respond.

The API-key configuration was also tested separately. The terminal test confirmed that the environment contained the configured API key and that the Gemini client could be initialized. The successful model test used the currently supported model configured during the final implementation.

14. Test Cases

TC ID	Test Scenario	Input	Expected Result	Observed Result
TC-01	Normal FAQ generation	Educational content about AI, ML and Deep Learning.	Security Status: ALLOWED and FAQs generated.	Passed – FAQs generated successfully.
TC-02	Prompt-injection security test	Input attempting to ignore previous instructions and reveal hidden system information.	Malicious prompt detected and request blocked before LLM processing.	Passed – Security Status: BLOCKED and patterns displayed.
TC-03	Empty input validation	No source content.	Input should be rejected with an appropriate validation message.	Designed validation case.
TC-04	Temporary AI service failure	Valid FAQ-generation request while model service is unavailable.	Application should handle the error without crashing and use the fallback where applicable.	Fallback/error-handling behaviour implemented.

15. Results and Discussion

The demonstrated implementation successfully covers the two primary project workflows. First, valid informational content passes the security layer and is converted into structured FAQs. Second, a prompt-injection example is identified by the security layer and blocked before it can reach the language model.

The screenshots provide visual evidence for both workflows. The normal-generation screenshot shows an ALLOWED status and successfully generated FAQ cards, while the security-test screenshot shows malicious prompt detection, blocking, detected patterns, and the final BLOCKED security decision.

16. Advantages

- Combines AI-based FAQ generation with a dedicated security layer.
- Provides a clear distinction between allowed and malicious input.
- Prevents demonstrated prompt-injection attempts from reaching the LLM.
- Displays security decisions in a user-friendly manner.
- Supports configurable FAQ quantity.
- Produces structured and readable FAQ results.
- Includes fallback/error handling for temporary external AI-service problems.
- Provides a practical academic demonstration of secure AI application design.

17. Limitations

- Pattern-based prompt-injection detection may not identify every possible attack formulation.
- The quality of AI-generated FAQs depends on the quality and clarity of the source content.
- External model availability can affect AI-based generation.
- A production system would require additional controls such as stronger policy enforcement, monitoring, logging, authentication, and comprehensive adversarial testing.

18. Future Enhancements

- Add multilingual FAQ generation and language selection.
- Add configurable FAQ styles such as beginner, academic, technical, and professional.
- Introduce stronger semantic prompt-injection detection in addition to pattern matching.
- Add audit logs for security events.
- Provide downloadable FAQ reports in PDF or Word format.
- Add authentication and role-based access control for production deployment.
- Add automated security regression tests containing diverse prompt-injection variants.

19. Conclusion

The Secure AI FAQ Generator demonstrates a complete secure AI workflow in which legitimate source content can be transformed into professional FAQs while suspicious prompt-injection attempts are identified and blocked. The application integrates input validation, prompt-injection detection, prompt isolation, malicious input blocking, security reporting, and FAQ generation within a single interface.

The successful FAQ-generation screenshot confirms that valid content is processed correctly, while the security-test screenshot confirms that a malicious prompt is detected and blocked before LLM processing. Therefore, the implemented project successfully demonstrates the intended combination of AI functionality and prompt-injection protection.

20. Overall Implementation Status

Component	Status	Evidence
FAQ Generator Interface	COMPLETED	Main interface screenshot
Valid Input Processing	COMPLETED	Security Status: ALLOWED
FAQ Generation	COMPLETED	FAQs generated successfully
Prompt-Injection Detection	COMPLETED	Malicious prompt detected
Malicious Input Blocking	COMPLETED	Request blocked by security layer
Security Status Reporting	COMPLETED	ALLOWED / BLOCKED status displayed
Detected Pattern Reporting	COMPLETED	Security patterns displayed

Security Decision	COMPLETED	Decision shown before LLM processing
-------------------	-----------	--------------------------------------